COURSE WEB STUDY

# JavaScript: من القيم إلى واجهة تتفاعل بوضوح

من المتغيرات والدوال إلى DOM وطلبات الشبكة ومشروع عملي صغير

JAVASCRIPT CORE

شرح عربي · أمثلة إنجليزية · تمارين عملية

# فهرس الكتاب

1 ابدأ بتجربة يمكن ملاحظتها ..... 3
2 افهم النوع قبل أن تكتب شرطاً ..... 4
3 اجعل الدالة مسؤولة عن شيء واحد ..... 4
4 نظّم القوائم والكائنات قبل أن تعرضها ..... 5
5 حدّث DOM من حالة واضحة ..... 5
6 اطلب البيانات وتعامل مع الفشل ..... 6
7 صحّح السبب ثم ابنِ مشروعاً صغيراً ..... 7
8 قائمة تسليم ذاتي ..... 7
9 مخطط تسليم المشروع الختامي ..... 8

## كيف تستخدم هذا الدليل؟

هذا كتاب ممارسة وليس قائمة أوامر. نفّذ كل مثال في ملف صغير، غيّر قيمة واحدة، ثم اكتب ما الذي تغيّر. الشرح عربي، لكن أسماء المتغيرات والكود بالإنجليزية كما ستراهما في المشاريع الحقيقية.

### نتائج التعلّم

- تمييز القيم والأنواع قبل معالجتها
- كتابة دوال صغيرة قابلة للاختبار
- تحديث DOM من حالة واضحة
- التعامل مع الشبكة والأخطاء دون كشف مفاتيح API

# 1 ابدأ بتجربة يمكن ملاحظتها

JavaScript تعمل في المتصفح أو على الخادم، لكن لا تبدأ بتطبيق كبير. أنشئ ملفاً واحداً، عرّف قيمة، واعرضها في Console. الهدف في أول جلسة هو أن تتأكد أن دورة: «اكتب ثم شغّل ثم لاحظ» تعمل عندك. كل تجربة صغيرة تقلل التخمين في الخطوات التالية.

**VALUE**
القيمة هي البيانات نفسها: نص أو رقم أو true وfalse أو كائن.

**STATE**
الحالة هي القيم التي تحدد ما الذي يجب أن تظهره الواجهة الآن.

```
const lessonTitle = 'JavaScript basics';
const studyMinutes = 25;
const isBreakTime = false;

console.log({ lessonTitle, studyMinutes, isBreakTime });
```

سمِّ القيم بحسب معناها، لا بحسب مكانها المؤقت. اسم مثل `studyMinutes` يخبرك بالوحدة والغرض، بينما `data` لا يخبرك بشيء. استخدم `const` للقيمة التي لن تُعاد إسنادها، و`let` فقط إذا كان التغيير جزءاً واضحاً من منطقك.

### تمرين: بيانات جلسة مذاكرة

عرّف عنواناً ومدة جلسة وحالة استراحة. اطبع الكائن في Console، ثم غيّر مدة الجلسة ولاحظ أن القيمة الرقمية ليست نصاً.

# 2 افهم النوع قبل أن تكتب شرطاً

المشكلة الشائعة ليست في if نفسها، بل في الافتراض الخاطئ عن القيمة التي تدخل إليها. بيانات النموذج أو الشبكة قد تكون نصاً حتى لو كانت تمثل رقماً. افحص المدخل أولاً، ثم اكتب المسار الطبيعي للحالة الصحيحة.

استخدم المقارنة الصارمة === كي لا يحول JavaScript الأنواع بصمت. وعندما تكون القيمة غير صالحة، أعد رسالة مبكرة من الدالة بدلاً من دفن الخطأ في شرط طويل. هذا يجعل القارئ يرى حالات الفشل قبل تفاصيل التنفيذ.

```
function formatMinutes(minutes) {
  if (typeof minutes !== 'number' || minutes < 0) {
    return 'قيمة مدة غير صالحة';
  }
  return `${minutes} دقيقة`;
}

console.log(formatMinutes(25));
```

### تمرين: حالة النجاح

اكتب دالة تستقبل درجة. إذا لم تكن رقماً أعد رسالة واضحة، وإذا كانت 50 أو أكثر أعد «ناجح»، وإلا أعد «راجع الدرس ثم أعد المحاولة». اختبر 49 و50 و'50'.

# 3 اجعل الدالة مسؤولة عن شيء واحد

الدالة الجيدة تستقبل مدخلاً معروفاً وتعيد نتيجة يمكن وصفها في جملة واحدة. دالة تحسب نسبة الإكمال لا يجب أن تبحث عن عناصر الصفحة أو تحفظ شيئاً في التخزين. افصل الحساب عن العرض، وستتمكن من اختبار المنطق في أي وقت حتى من دون متصفح.

إذا طال اسم الدالة أو احتاجت عشر قيم خارجية كي تعمل، توقف. غالباً عليك تمرير قيمة كـ parameter أو فصل خطوة أصغر. أسماء مثل completionPercent أو renderProgress تقول بوضوح ما الذي تفعله الدالة.

```
function completionPercent(done, total) {
  if (total === 0) return 0;
  return Math.round((done / total) * 100);
}

console.log(completionPercent(3, 5)); // 60
```

### تمرين: دالة متبقية

اكتب دالة `remainingLessons` تستقبل إجمالي الدروس والمكتمل منها، وتعيد عدداً صحيحاً لا يقل عن صفر.

اكتب اختباراً لحالة إجمالي صفر.

# 4 نظّم القوائم والكائنات قبل أن تعرضها

استخدم المصفوفة عندما تمثل مجموعة مرتبة مثل الدروس، واستخدم الكائن لوصف كيان واحد مثل درس محدد. لا تخلط بينهما لأن نوع البنية يحدد كيف تبحث وتحدّث وتعرض البيانات. القراءة الجيدة للبيانات تسبق أي واجهة جميلة.

`map` تحوّل كل عنصر إلى قيمة جديدة، و`filter` تختار عدة عناصر تطابق شرطاً، و`find` يعيد أول عنصر واحد. لا تستخدم طريقة لأن اسمها مألوف؛ اخترها بحسب الناتج الذي تريده.

```
const lessons = [
  { id: 'html', title: 'HTML', done: true },
  { id: 'css', title: 'CSS', done: false },
];

const remainingTitles = lessons
  .filter((lesson) => !lesson.done)
  .map((lesson) => lesson.title);
```

**تمرين: قائمة غير مكتملة**

أنشئ ثلاثة دروس في مصفوفة كائنات. اعرض عناوين الدروس غير المكتملة فقط، ثم أضف درساً جديداً من دون تعديل العناصر السابقة.

# 5 حدّث DOM من حالة واضحة

DOM هو تمثيل الصفحة داخل JavaScript. بدلاً من كتابة نص جديد في أماكن متفرقة بعد كل نقرة، احتفظ بحالة صغيرة ثم اجعل دالة `render` واحدة تقرر ماذا يظهر. عند حدوث حدث، حدّث الحالة ثم استدعِ `render`.

استخدم `textContent` للنص العادي. إدخال محتوى المستخدم في `innerHTML` بلا تعقيم ليس اختصاراً آمناً، وقد يفتح باباً لسلوك غير مطلوب. اختر العناصر مرة واحدة وامنحها أسماء توضح دورها.

```
let completed = 0;
const status = document.querySelector('[data-status]');
const button = document.querySelector('[data-complete]');

function render() {
```

```
  status.textContent = `تمت ${completed} دروس`;
}

button?.addEventListener('click', () => {
  completed += 1;
  render();
});
render();
```

**تمرين: زر الإكمال**

أنشئ صفحة فيها زر وعنصر حالة. اجعل كل ضغطة تزيد العداد، ثم أعد تحميل الصفحة وحدد ما الذي سيختفي ولماذا.

# 6 اطلب البيانات وتعامل مع الفشل

طلب الشبكة ليس مضموناً: قد يتأخر أو يرجع حالة خطأ أو يفشل الاتصال. async وawait تجعل التسلسل مقروءاً، لكنهما لا تلغيان الحاجة إلى try/catch أو فحص response.ok. صمّم الواجهة لحالات التحميل والفشل قبل وجود endpoint حقيقي.

الأسرار مثل مفاتيح مزود الذكاء الاصطناعي لا توضع في JavaScript يعمل في المتصفح. احفظها في الخادم، واجعل الواجهة تطلب بيانات آمنة عبر مسار API. هذه قاعدة أمان وليست تفضيلاً تنظيمياً.

```
async function loadStudyDomains() {
  try {
    const response = await fetch('/learning-api/study-domains');
    if (!response.ok) throw new Error('Request failed');
    return await response.json();
  } catch {
    return { error: 'تعذر تحميل المسارات الآن.' };
  }
}
```

**تمرين: حالة خطأ مفيدة**

اكتب دالة طلب تعرض رسالة تحميل، ثم رسالة عربية عند الفشل. غيّر المسار عمداً مرة لتتأكد أن المستخدم لا يرى صفحة فارغة.

# 7 صحّح السبب ثم ابنِ مشروعاً صغيراً

الـ bug دليل على مسار لم تفهمه بعد. أعد إنتاجه، قلّل المثال، اقرأ الرسالة، ثم غيّر شيئاً واحداً. استخدم `console.assert` للحالات التي تعرف نتيجتها، واحذف رسائل التصحيح المؤقتة عندما ينتهي دورها.

المشروع الختامي هو قائمة مذاكرة تفاعلية: دروس، زر إكمال، نسبة تقدّم، وحفظ محلي بسيط بعد أن تعمل النسخة في الذاكرة. اكتب الحالات الحدية: قائمة فارغة، كل الدروس مكتملة، أو تخزين قديم تالف. هذه الحالات هي الفرق بين demo وتطبيق يمكن الاعتماد عليه.

```
console.assert(completionPercent(1, 2) === 50);
console.assert(completionPercent(0, 0) === 0);

const state = {
  lessons: [{ id: 'js', title: 'JavaScript', done: false }],
};
```

**تمرين: قائمة مذاكرة تفاعلية**

نفّذ المشروع من أربع مراحل: بيانات أولية، render، أحداث الأزرار، ثم localStorage. اكتب README قصيراً يشرح طريقة التجربة والحالات التي اختبرتها.

# 8 قائمة تسليم ذاتي

- هل تستطيع وصف نوع كل قيمة قبل معالجتها؟
- هل دوال الحساب مستقلة عن DOM؟
- هل تظهر الواجهة حالة تحميل وخطأ مفهومة؟
- هل أبقيت أسرار API على الخادم؟
- هل اختبرت إجمالي صفر وقائمة فارغة؟

# 9 مخطط تسليم المشروع الختامي

حوّل قائمة المذاكرة إلى تسليم صغير يمكن لأي شخص تجربته. ابدأ بملف `index.html` فيه منطقة للحالة وقائمة وزر واضح. أضف ملف `app.js` يحوي بيانات البداية ودالة `render` ودوال الأحداث. لا تضف التخزين إلا بعد أن تعمل التجربة كلها في الذاكرة.

**DATA · 01**

عرّف `state.lessons` مع id وعنوان وحالة إكمال. جرّب قائمة فارغة ودرساً واحداً.

**RENDER · 02**

حوّل الحالة إلى عناصر ظاهرة. اجعل `render` هي المكان الوحيد الذي يكتب في واجهة القائمة.

**EVENTS · 03**

اربط كل زر بـ id محدد، ثم حدّث الحالة وأعد الرسم. لا تنشئ listener داخل render.

**PERSIST · 04**

بعد نجاح المسارات السابقة فقط، احفظ نسخة JSON محمية بـ try/catch في localStorage.

اختبر بوضوح: افتح الصفحة على عرض 375px، أكمل كل الدروس، أعد تحميلها، ثم امسح التخزين المحلي. اكتب نتيجة كل اختبار في README. هذه الخطوة تحوّل التمرين من كود يعمل بالصدفة إلى أساس يستطيع شخص آخر مراجعته وتشغيله.